NAME: C. PRIYANKA

REG NO: 192511074

29) Prolog Program for Forward Chaining

Aim

To write a Prolog program to implement Forward Chaining and execute the required queries.

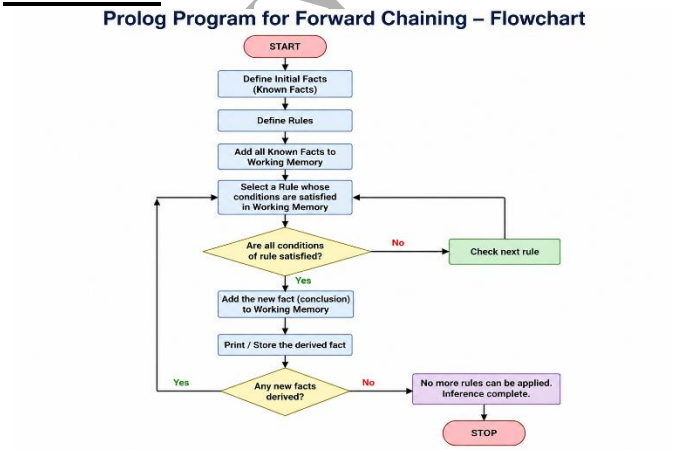
Objectives

- To understand forward chaining.
- To derive new facts from existing facts and rules.
- To implement rule-based reasoning in Prolog.

Algorithm

1. Define the initial facts.
2. Define rules based on the available facts.
3. Start with the known facts.
4. Apply the rules whose conditions are satisfied.
5. Add the newly derived facts.
6. Continue until no new facts can be derived.
7. Query the required facts.

Flow Chart



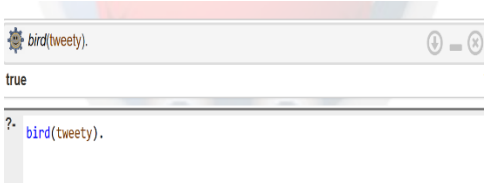
Prolog Code

```
% Facts
bird(tweety).
bird(robin).
has_feathers(tweety).
has_feathers(robin).

% Rules
feathered(X) :- has_feathers(X).
can_fly(X) :- bird(X), feathered(X).

% Queries
```

Sample Output



Result

Thus, the Prolog program for Forward Chaining was successfully implemented and the required queries were executed.

Conclusion

Forward chaining derives new information by starting from known facts and applying applicable rules.

30) Prolog Program for Backward Chaining

Aim

To write a Prolog program to implement Backward Chaining and execute the required queries.

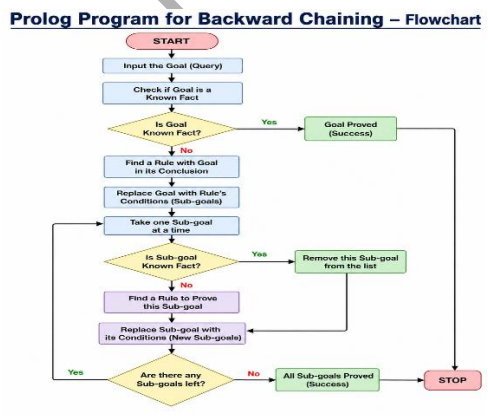
Objectives

- To understand backward chaining.
- To prove a goal using available rules and facts.
- To implement goal-driven reasoning in Prolog.

Algorithm

1. Define facts and rules.
2. Start with the required goal.
3. Check whether the goal is directly available as a fact.
4. If not, find a rule that can produce the goal.
5. Treat the rule's conditions as sub-goals.
6. Continue until the sub-goals are satisfied.
7. Return true if all conditions are satisfied.

Flow Chart



Prolog Program

```
% Facts
bird(tweety).
has_wings(tweety).
has_feathers(tweety).

% Rules
can_fly(X) :- bird(X), has_wings(X), has_feathers(X).

% Queries
```

Sample Output

Result

Thus, the Prolog program for Backward Chaining was successfully implemented.

Conclusion

Backward chaining starts with a goal and works backward to determine whether the goal can be proved using facts and rules.

31) Create a Web Blog Using WordPress

Aim

To create a simple Web Blog using WordPress demonstrating Anchor Tag, Title Tag, headings, images and hyperlinks.

Objectives

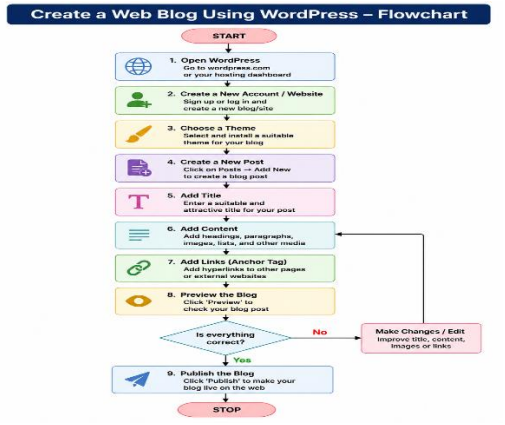
- To understand the basic structure of a web blog.
- To create pages and posts using WordPress.
- To demonstrate HTML elements such as title and anchor tags.

Algorithm

1. Open WordPress.
2. Create a new website/blog.
3. Select a suitable theme.
4. Create a new blog post.
5. Add a suitable title.
6. Add headings, paragraphs and images.

7. Add hyperlinks using the anchor tag.
8. Preview the blog.
9. Publish the blog.

Flow Chart



Code

```
 <!DOCTYPE html>
<html>
<head>
   <title>My College Blog</title>
</head>

<body>

<h1>Welcome to My College Blog</h1>

<h2>About My College</h2>

<p>
This blog contains information about college activities,
events and student life.
</p>

<a href="https://www.example.com">
Visit Website
</a>

</body>
</html>
```

Sample Out put

```
-----------------------------------
      MY COLLEGE BLOG
-----------------------------------

Welcome to My College Blog

About My College

This blog contains information about
college activities, events and student life.
```

Result

Thus, a simple web blog was successfully created using WordPress with basic web elements.

Conclusion

WordPress provides an easy platform for creating and publishing blogs, while HTML tags help structure web content.

32) Prolog Program to Implement Pattern Matching

Aim

To write a Prolog program to implement **pattern matching**.

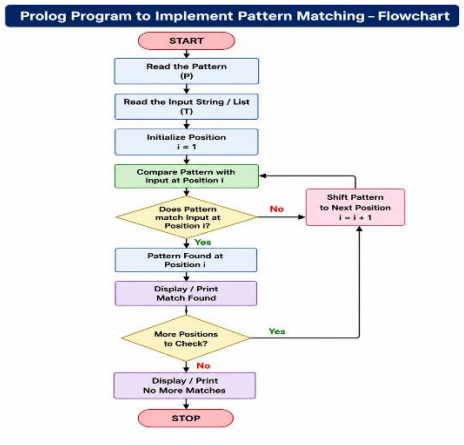
Objectives

- To understand pattern matching in Prolog.
- To compare patterns with input data.
- To use Prolog's unification mechanism.

Algorithm

1. Define a pattern.
2. Define input data.
3. Compare the pattern with the input.
4. Prolog attempts to unify both terms.
5. If matching is possible, return the matching result.
6. Otherwise, return false.

FlowChart

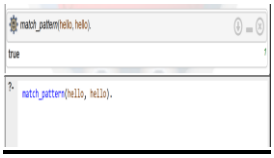


Prolog Code

```
match_pattern(X, X).


check :-
   match_pattern(hello, hello).
```

Sample Output



Result

Thus, the Prolog program for **pattern matching** was successfully implemented.

Conclusion

Pattern matching in Prolog is performed using **unification**, which automatically finds matching values for variables.

33. Prolog Program to Find the Number of Vowels

Aim

To write a Prolog program to find the **number of vowels** present in a given string.

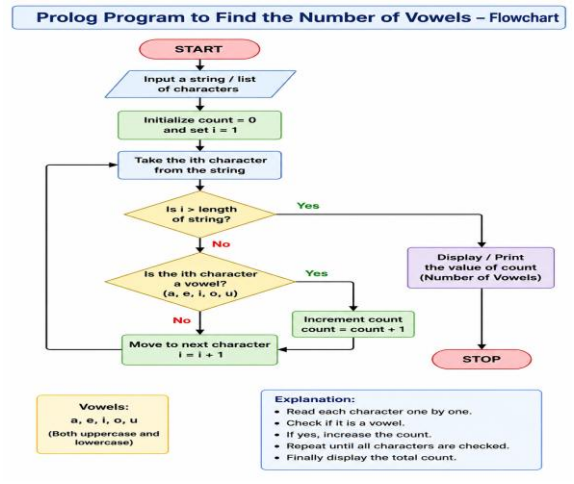
Objectives

- To identify vowels in a string.
- To count the total number of vowels.
- To understand recursion and list processing in Prolog.

Algorithm

1. Read a string as a list of characters.
2. Check each character.
3. If the character is a vowel, increase the count.
4. If it is not a vowel, continue to the next character.
5. Repeat until the list becomes empty.
6. Display the total number of vowels.

Flow Chart



Prolog Code

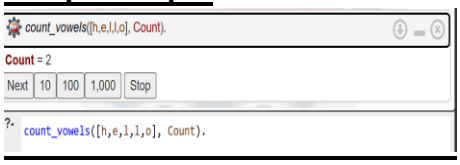
```
vowel(a).
vowel(e).
vowel(i).
vowel(o).
vowel(u).

count_vowels([], 0).

count_vowels([H|T], Count) :-
    vowel(H),
    count_vowels(T, C),
    Count is C + 1.

count_vowels([H|T], Count) :-
    \+ vowel(H),
    count_vowels(T, Count).
```

Sample Output



Result

Thus, the Prolog program successfully finds and displays the number of vowels in a given list of characters.

Conclusion

The program demonstrates the use of facts, recursion, lists and arithmetic operations in Prolog to count vowels.